Prof. Dr. Gogol-Döring

Theoretische Informatik 1

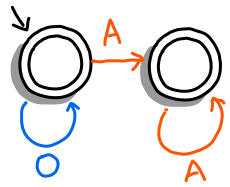
Lösungen zu den Klausur Übungsaufgaben

▸ Lösungen zu Aufgabe 1

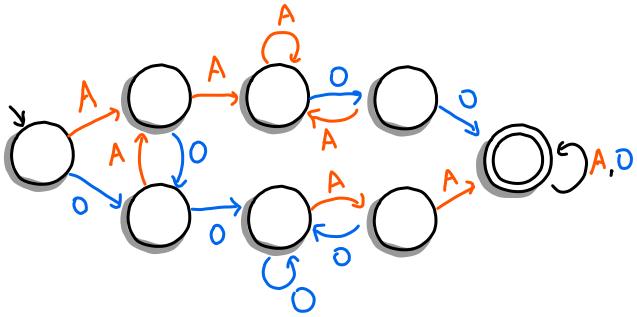
1. *DEA:* $\delta: Q \times \Sigma \rightarrow Q$
2. *NEA:* $\delta: Q \times \Sigma \rightarrow \mathcal{P}(Q)$
3. *ε-NEA:* $\delta: Q \times \{\Sigma \cup \{\varepsilon\}\} \rightarrow \mathcal{P}(Q)$

▸ Lösungen zu Aufgabe 2

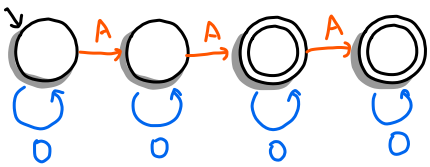
1. L = {w ∈ Σ* | in w folgt niemals ein "O" auf ein "A"}



2. L = {w ∈ Σ*| w enthält sowohl den String "AA" als auch den String "OO"}

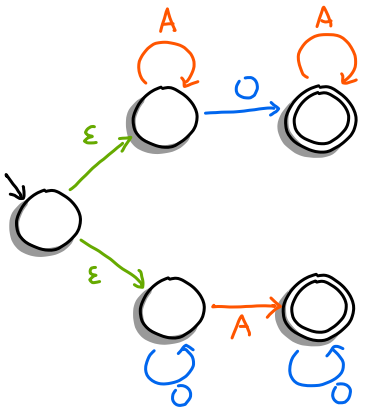


3. L = {w ∈ Σ*| w enthält den Buchstaben "A" entweder genau zweimal oder genau dreimal}



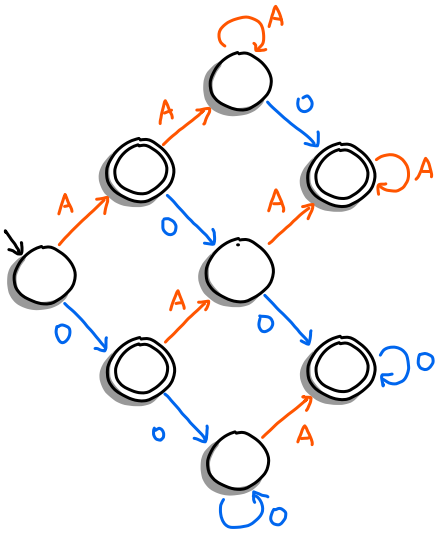
▸ Lösungen zu Aufgabe 3

1. L = {w ∈ {A, O}* | w enthält genau ein "A" <u>oder</u> genau ein "O"}



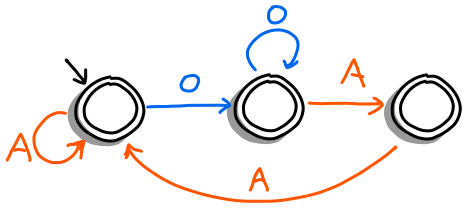
A*OA*|O*AO*

2. L = {w ∈ {A, O}* | <u>entweder</u> enthält w genau ein "A" <u>oder</u> genau ein "O"



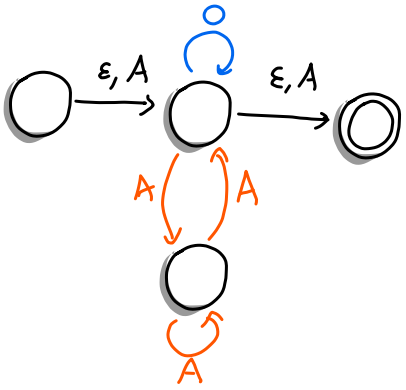
O | OAAA* | AOAA* | AAA*OA* |
A | AOOO* | OAOO* | OOO*AO*

3. L = {w ∈ {A, O}* | w enthält <u>nicht</u> den Teilstring "OAO"}



(A|OO*AA)*O*(A|ε)

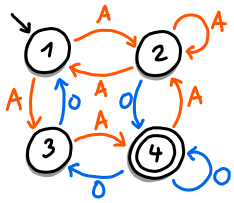
oder auch:



(ε|A)(O|AA*A)*(ε|A)

▸ Lösungen zu Aufgabe 4

1. AOAO: 1→2→4→2→4
2. AAAAOAA: 1→2→2→1→3→1→3→4
3. AAOOOA: 1→3→4→4→4→3→4



▸ Lösungen zu Aufgabe 5

1. L = {w ∈ {A, O}* | w beginnt mit A gefolgt von beliebig vielen Paaren AA und OO}
   = A(AA|OO)*

2. L = (AO*A)*

3. L = {w ∈ {A, O}* | w beginnt mit beliebig vielen A gefolgt von beliebig vielen O gefolgt von einem A}
   =A*O*A

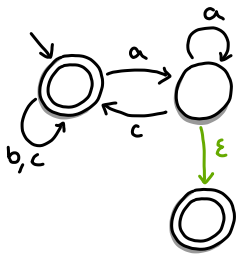
4. L = {w ∈ {x, y}* | w enthält xx}
   = (x|y)*xx(x|y)*

5. L = {w ∈ {A, O}* | der erste und der letzte Buchstabe von w sind gleich}
   = A(A|O)*A | O(A|O)*O

6. L = {w ∈ {A, B}* | w endet mit A}
   = (A|B)*A

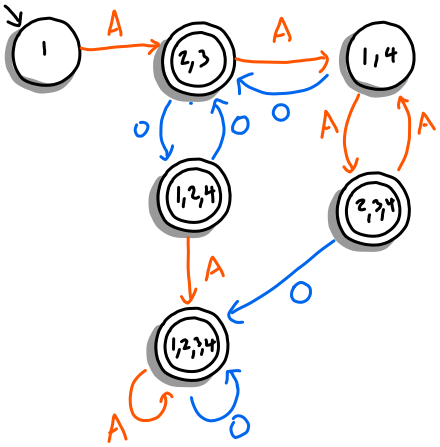
7. L = {w ∈ {a, b, c}* | w enthält nicht den Teilstring ab}

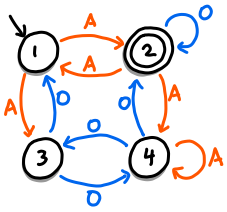
Anmerkung: Die beiden rechten Zustände sind nicht produktiv, d.h. man kann den Automaten wie folgt vereinfachen:



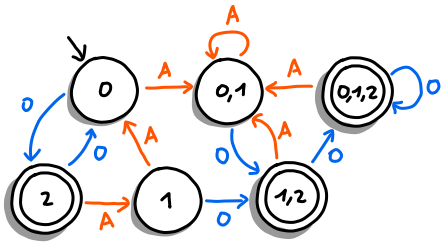
▸ Lösungen zu Aufgabe 6

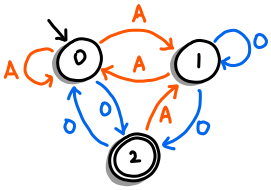
1.





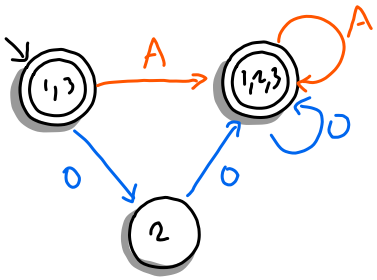
2.

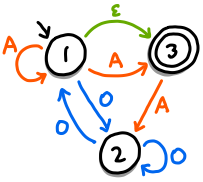




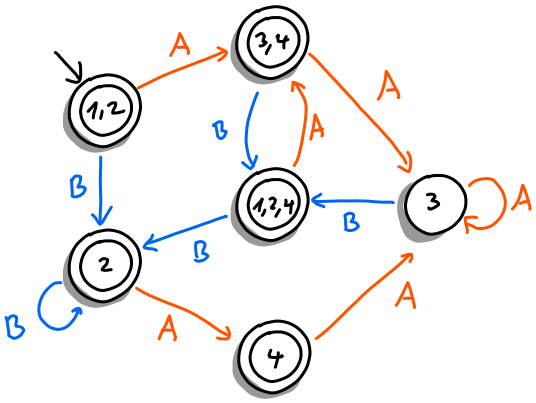
▸ Lösungen zu Aufgabe 7

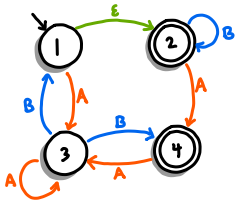
1.





2.





▸ Lösungen zu Aufgabe 8

Wenn der String von der Grammatik erzeugt werden kann, beweisen wir das, indem wir den String entsprechend einfärben. Regulärer Ausdruck: (*ABB**|**ABA*AA**)*B

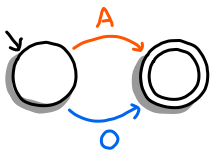
1. nein
2. ja: **ABAAAA** *ABB* B
3. ja: *AB* **ABAA** B
4. nein
5. ja: **ABAA** *ABB AB* B

▸ Lösungen zu Aufgabe 9

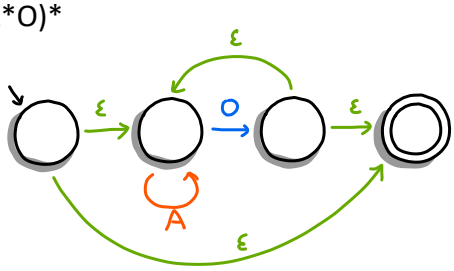
1. L = Σ*AΣ
2. L = Σ*AAΣ*
3. L = AΣ*O | OΣ*A
4. L = (A | OOO*)*
5. L = (O*AO*AO*)*

▸ Lösungen zu Aufgabe 11

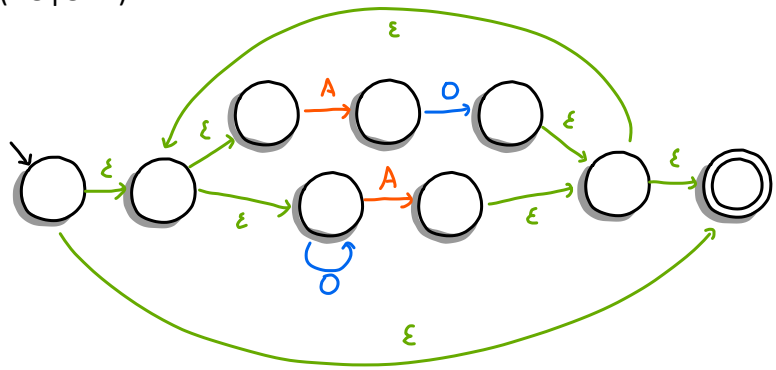
1. Ø|A|O = A|O



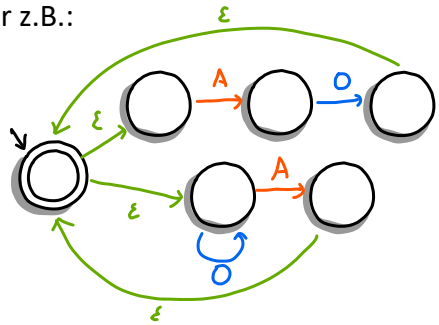
2. (A*O)*



3. (AO|O*A)*

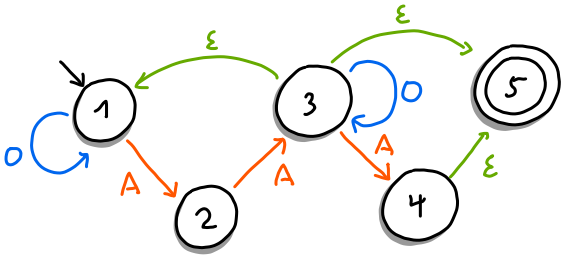


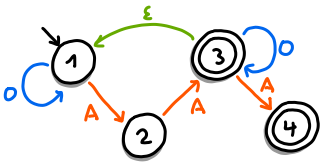
oder z.B.:

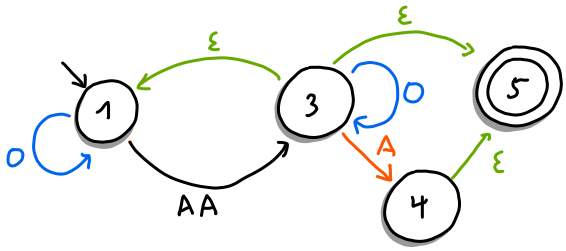


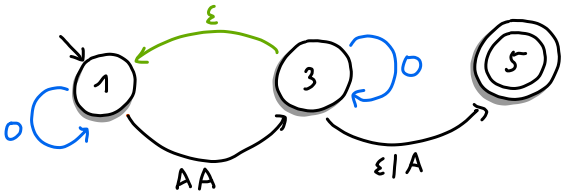
▸ Lösung zu Aufgabe 12

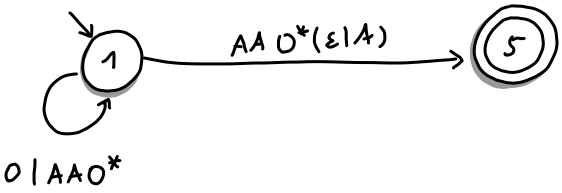
1.





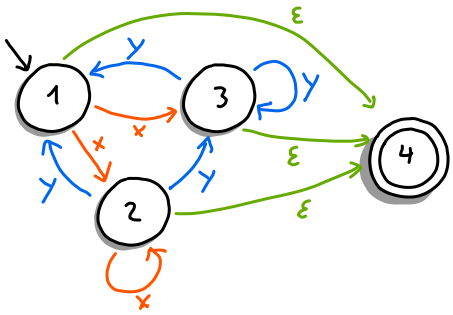


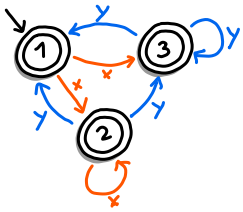


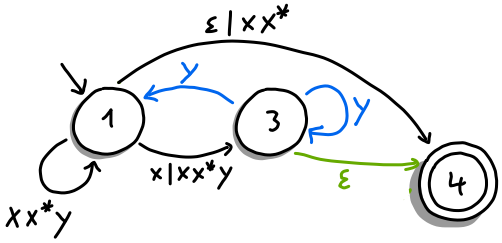


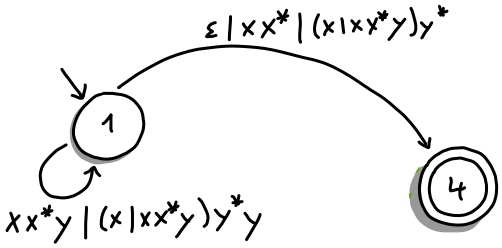
regulärer Ausdruck: (O|AAO*)*AAO*(ε|A)

2.



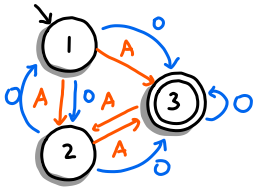






regulärer Ausdruck:
(xx*y|(x|xx*y)y*y)*(ε|xx*|(x|xx*y)y*)

▸ Lösung zu Aufgabe 13



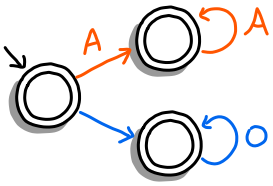
1. w = AAOA
a = ε, b = AAO, c = A

2. w = AOA
a = ε, b = AO, c = A

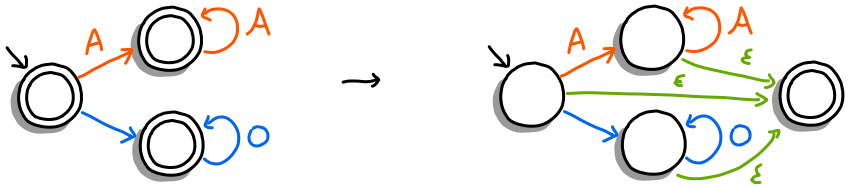
3. w = AO
a = A, b = O, c = ε

▸ Lösung zu Aufgabe 14

1. L = A*|O*



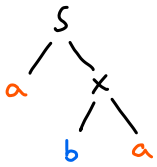
2. Man kann alle Zustände, die akzeptierend sein sollen, durch nicht-akzeptierende Zustände ersetzen, von denen aus es je einen ε-Übergang zu einem einzelnen akzeptierenden Endzustand gibt, z.B.:



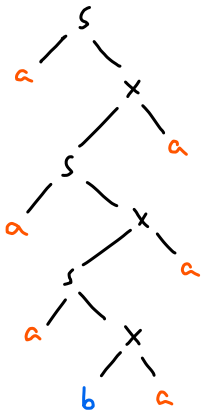
▸ Lösung zu Aufgabe 15

S → aX | SXb
X → ba | Sa

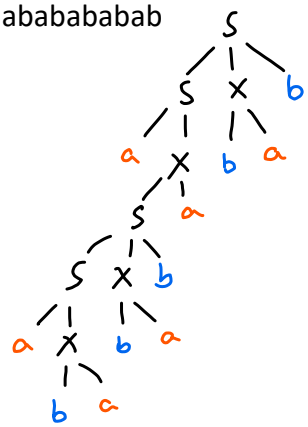
1. aba



2. aaabaaa



3. aababababab



▸ Lösungen zu Aufgabe 16

1. S → ε | aS | Sbb

L = $\{a^n b^{2n} \mid n \in \mathbb{N}\}$

2. S → b | SaS

L = {w ∈ {a, b}* | w beginnt mit b und endet mit b und in w kommen niemals zwei gleiche Buchstaben direkt hintereinander}
= b(ab)*

3. S → bX | Sa
X → ε | Xb

L = {w ∈ {a, b}* | w beginnt b, eventuell gefolgt von beliebig vielen b, eventuell gefolgt von beliebig vielen a}
= bb*a*

▸ Lösung zu Aufgabe 17

1. L = {w ∈ {a, b}* | w beginnt und endet mit a}

S → aXa | a
X → aX | bX | ε

2. L = $\{a^n b a^m \mid n, m \in \mathbb{N}_0 \text{ und } n \geq m\}$

S → aSa | Xb
X → aX | ε

3. (ab|b|c)*

S → abS | bS | cS | ε

▸ Lösung zu Aufgabe 18

⟨Ausruck⟩ ::= ⟨Variable⟩ |
⟨Ausdruck⟩ " *and* " ⟨Ausdruck⟩ |
⟨Ausdruck⟩ " *or* " ⟨Ausdruck⟩ |
"*not* " ⟨Ausdruck⟩ |
"(" ⟨Ausdruck⟩ ")"

▸ Lösung zu Aufgabe 19

Gegeben ist folgende Grammatik:

S → a | aXS | YaX | Xb | ZY
X → ScS | ZaX| bb
Y → cZ | YSc | ZYa
Z → XbY | XcZ

Beobachtung: Y und Z sind nicht produktiv ⇒ wir können sie aus der Grammatik entfernen

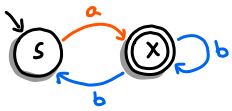
S → a | aXS | Xb
X → ScS | bb

Nun ersetzten wir das X in den Produktionen von S mit den Produktionen von X:

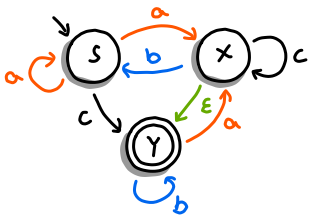
S → a | aScSS | abbS | ScSb | bbb

▸ Lösung zu Aufgabe 20

1. S → aX
   X → bX | bS | ε

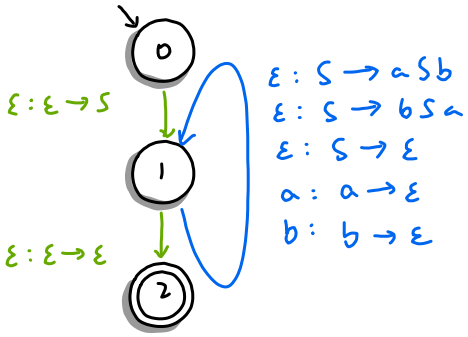


2. S → aS | aX | cY
   X → bS | cX | Y
   Y → aX | bY | ε

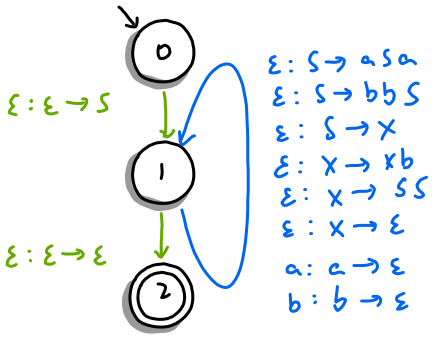


▸ Lösung zu Aufgabe 21

1. S → aSb | bSa | ε

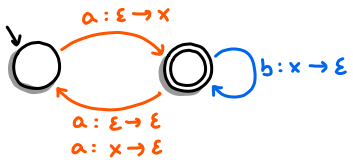


2. S → aSa | bbS | X
   X → Xb | SS | ε

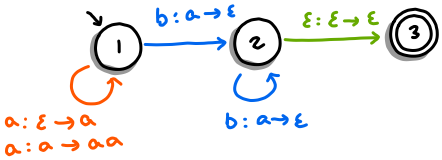


▸ Lösung zu Aufgabe 22

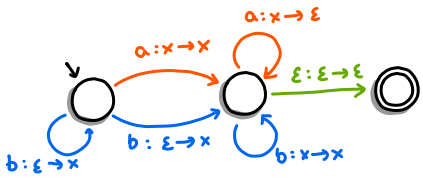
1. L = (a(ε|b)a)*a(ε|b)



2. L = {$a^n b^n$ | n ∈ ℕ}
   Achtung: ε ∉ L, da der Automat nur in Zustand 2 wechseln kann, wenn er mindestens ein b von der Eingabe liest.



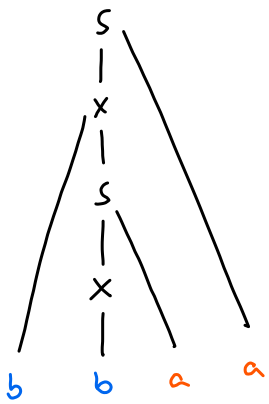
3. L = (ba|b)b*a



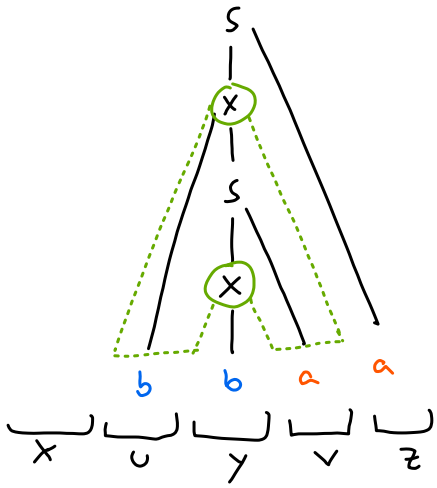
▸ Lösung zu Aufgabe 23

S → abSX | Xa
X → Xaa | bS | b

Zeichnen Sie einen Ableitungsbaum, in dem es einen Pfad von S bis zu einem Blatt gibt, der Länger ist als die Anzahl der Nichtterminalsymbole + 1 (in diesem Fall also länger als 3), also z.B.



Wählen Sie ein Nichtterminal, das auf diesem Pfad doppelt vorkommt, also z.B. X, und Trennen Sie den Baum entsprechend auf:



Alles, was zwischen den beiden Vorkommen von X links bzw. rechts an Terminalen produziert wird, wird Teil von u bzw. v. Alle restlichen Terminale teilen sich auf x, y und z auf:

⇒ x = ε, u = "b", y = "b", v = "a", z = "a"
w = xuyvz = "bbaa"

▸ Lösung zu Aufgabe 24

1. L = {w ∈ Σ* | die Länge von w ist ungerade oder w beginnt mit "A"}

2. L = {w ∈ Σ* | w enthält gleich viele a wie b wie c}

3. L = {w ∈ Σ* | w beginnt mit "A"}
Achtung: Die *loop*-Schleife wird beim ersten Durchlauf bereits abgebrochen ("plötzlicher Schleifentod").

▸ Lösung zu Aufgabe 25

1. L = {w ∈ {a, b, c}* | in w kommen niemals zwei gleiche Buchstaben direkt hintereinander}

```
n ← len(in)
if n ≤ 1
    return true
end
for i ← 0, …, n−2
    if in[i] = in[i+1]
        return false
    end
end
return true
```

2. L = {w ∈ Σ* | die Länge von w ist eine Primzahl}

```
n ← len(in)
if n < 2
    return false
end
if n = 2
    return true
end
for i ← 2, …, n − 1
    if n % i = 0
        return false
    end
end
return true
```

3. L = {w ∈ {a, b}* | Es gibt ein n ∈ ℕ so dass w = $u_1u_2...u_n$ mit $u_i = a^ib^i$}

```
n ← len(in)
counta = 0        ⇦ Anzahl der gelesenen a, zu denen noch kein b gelesen worden ist
reada = true      ⇦ reada = true, wenn gerade a gelesen werden, sonst false
if n = 0
   return false
end
for i ← 0, ..., n−1
   if in[i] = "a"
      if reada
         ++counta
      else
         if counta
            return false   ⇦ nicht genügend b gelesen, bevor das nächste a gekommen ist
         else
            reada ← true   } ein neues u_i hat angefangen
            counta ← 1     }
         end
      end
   else
      reada ← false
      if counta
         −−counta
      else
         return false   ⇦ zu viele b gelesen
      end
   end
end
if counta
   return true
else
   return false
end
```

▶ Lösung zu Aufgabe 26

```
k ← 0
while n
   n ← n · (n−5)
   −−n
   ++k
end
return k
```

Definition:

$$\text{Rückgabewert} = \begin{cases} 0 & \text{, wenn } n = 0 \\ 1 & \text{, wenn } 1 \le n \le 5 \\ 2 & \text{, wenn } n = 6 \\ \text{undefiniert} & \text{, wenn } n \ge 7 \end{cases}$$

Erklärung:

- für n = 0 hält das Programm an, ohne dass die *while*-Schleife durchlaufen wird ⇒ Rückgabewert 0
- für 0 < n ≤ 5 wird die *while*-Schleife genau einmal durchlaufen; dabei wird n auf 0 gesetzt; anschließend hält das Programm an ⇒ Rückgabewert 1
- für n = 6 wird n beim ersten Durchlauf der *while*-Schleife auf 5 gesetzt, beim zweiten Durchlauf der while-Schleife auf 0; anschließend hält das Programm an ⇒ Rückgabewert 2
- für n ≥ 7 wächst n bei jedem Durchlauf der *while*-Schleife; das Programm hält nicht an.

▸ Lösung zu Aufgabe 27

```
n ← count(a)
if n ≤ 1
   return false
end
for i ← 0, …, n−1
   for j ← 0, …, n−1
      if (i ≠ j) and (x = a_i + a_j)
         return true
      end
   end
end
return false
```

oder etwas eleganter:

```
n ← count(a)
if n ≤ 1
   return false
end
for i ← 0, …, n−2
   for j ← i+1, …, n−1
      if x = a_i + a_j
         return true
      end
   end
end
return false
```

▸ Lösung zu Aufgabe 28

```
while true
   if n > 10
      n ← n − 3
   else
      n ← n / 2
   end
   if n < 3
      return n + 2
   end
end
```

```
L1:   if n > 10
         n ← n − 3
         goto L2
      else
         n ← n / 2
L2:   if n < 3
         n ← n + 2
      else
         goto L1
      end
   end
   return n
```

▸ Lösungen zu Aufgabe 29

1. `func1(n) = n > 1 ? func1(n−2) + 1: 0`

func1(7)
= func1(5) + 1
= func1(3) + 1 + 1
= func1(1) + 1 + 1 + 1
= 0 + 1 + 1 + 1 = **3**

2. `func2(n) = n > 1 ? func2(n−1) · func(n−2) : n+1`

func2(0) = 1
func2(1) = 2
func2(2) = func2(1) · func2(0) = 2
func2(3) = func2(2) · func2(1) = 4
func2(4) = func2(3) · func2(2) = 8
func2(5) = func2(4) · func2(3) = 32
func2(6) = func2(5) · func2(4) = 256
func2(7) = func2(6) · func2(5) = **8192**

3. `func3(n) = n ≥ 10 ? n+1 : func3(func3(n+1))`

func3(7)
= func3(func3(8))
= func3(func3(func3(9)))
= func3(func3(func3(func3(10))))
= func3(func3(func3(11)))
= func3(func3(12))
= func3(13) = **14**

▸ Lösung zu Aufgabe 30

1. `language1(a) = len(a) % 2 ? a[len(a)/2] = "A" : false`

L = {w ∈ Σ* | w hat eine ungerade Länge und der Buchstabe in der Mitte von w ist A}
= {$\Sigma^n A \Sigma^n$ | n ∈ $\mathbb{N}_0$}

2.
```
language2(a) = len(a) % 2 ? false : L2(a, len(a)−1)
L2(a, i) = (a[i] = "A") and (i > 1 ? L2(a, i−2) : true)
```

L = {w ∈ Σ* | w hat eine gerade Länge, und jeder zweite Buchstabe ist ein A}
L = (ΣA)*

3.

```
language3(a) = L3(a, len(a))
L3(a, n) = n % 3 ? false : match(a, n/3, 0, n/3) and match(a, n/3, n/3, 2·n/3)
match(a, m, i, j) = m ? (a[i] = a[j]) and match(a, m−1, i+1, j+1) : true
```

L = {www | w ∈ Σ*}

▸ Lösungen zu Aufgabe 31

1. L = Σ*aΣ*

```
language1(a) = needA(a, 0)
needA(a, i) = i < len(a) ? (a[i] = "A" ? true : needA(a, i+1)) : false
```

2. L = {w ∈ Σ* | w enthält mindestens so viele a wie b}

```
language2(a) = countChar(a, "A", 0) ≥ countChar(a, "B", 0)
countChar(a, c, i) = i < len(a) ? countChar(a, c, i+1) + (a[i] = c) : 0
```

3. L = (b|c)* | Σ*a(a|c)*

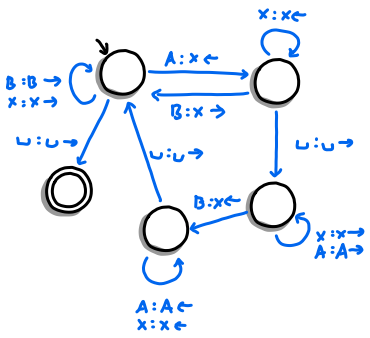
```
language3(a) = searchA(a, 0)
searchA(a, i) = i < len(a) ? (a[i] = "A" ? noC(a, i+1) : searchA(a, i+1)) : true
noC(a, i) = i < len(a) ? (a[i] = "C" ? false : noC(a, i+1)) : true
```

▸ Lösung zu Aufgabe 32

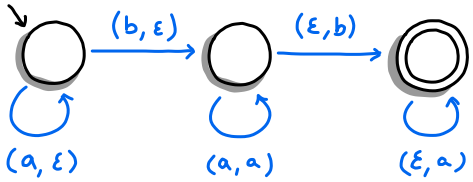
```
largestDivisor(n): findDivisor(n, n−1)
findDivisor(n, m): n % m ? findDivisor(n, m−1) : m
```

▸ Lösung zu Aufgabe 33

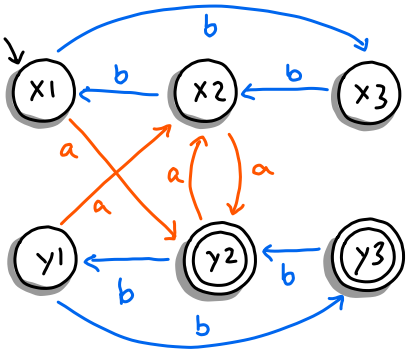
L = {w ∈ Σ* | w enthält mindestens so viele B wie A}



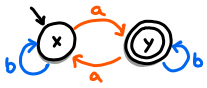
▸ Lösung zu Aufgabe 34



▸ Lösung zu Aufgabe 35



A_1



A_2

